Lista 9

Zadanie 1 (1 punkt). W pewnej fikcyjnej grze planszowej, każdy gracz posiada pewną ilość *pieniędzy*, *domków*[1], oraz *żetonów budowy*. Wszystkie te dane są wyrażone jako liczby naturalne. Napisz klasę `Player`, której instancja reprezentuje gracza (i przechowuje opisane wyżej dane), wyposażoną w metody:

- `__init__(self, money=1000, houses=0, tokens=1)` – inicjalizator, który dla liczb naturalnych `money`, `houses` i `tokens` tworzy gracza posiadającego `money` pieniędzy, `houses` domków i `tokens` żetonów budowy.
- `add_tokens(self, tokens=1)`, gdzie `tokens` to liczba naturalna, która dodaje `tokens` żetonów budowy graczowi `self`.
- `change_money(self, money)`, gdzie `money` to liczba całkowita (może być ujemna), która zmienia stan pieniędzy gracza `self` o wartość `money`.
- `build_houses(self, n=1)`, która graczowi `self` dodaje `n` domków kosztem 400`n` pieniędzy i `n` żetonów budowy.

Gdyby wywołanie którejkolwiek metody miało doprowadzić do niedozwolonego stanu gracza (np. ujemnej liczby żetonów konstrukcji), metoda powinna rzucić wyjątek `ValueError` i nie zmieniać stanu gracza.

Zadanie 2 (1,5 punktu). Napisz **własną**[2] klasę `Complex`, której instancje reprezentują liczby zespolone. Zaimplementuj następujące metody specjalne[3]:

- `__init__(self, real=0, imag=0)` – inicjalizator tworzący liczbę zespoloną postaci `real` + `imag`$\cdot i$.
- `__abs__(self)` – zwraca liczbę odpowiadającą |`self`|.
- `__neg__(self)` – zwraca liczbę odpowiadającą −`self`.
- `__str__(self)` – zwraca napis reprezentujący `self`.
- `__add__(self, other_num)` – zwraca liczbę odpowiadającą `self` + `other_num`.
- `__sub__(self, other_num)` – zwraca liczbę odpowiadającą `self` − `other_num`.
- `__mul__(self, other_num)` – zwraca liczbę odpowiadającą `self` · `other_num`.
- `__truediv__(self, other_num)` – zwraca liczbę odpowiadającą `self` / `other_num`.
- `__eq__(self, other_num)}` – sprawdza, czy `self` = `other_num`.

Zaimplementuj też metodę `conjugate(self)`, zwracającą liczbę odpowiadającą sprzężeniu `self`.

Zadanie 3 (1,5 punktu). Napisz klasę `Triangle`, której instancja reprezentuje domknięty trójkąt na płaszczyźnie (tzn. zbiór punktów ograniczony trójkątem, wraz z jego brzegiem), zawierającą następujące metody:

- `__init__(self, x1, y1, x2, y2, x3, y3)` – inicjalizator tworzący trójkąt, którego wierzchołki (w kolejności przeciwnej do ruchu wskazówek zegara) to kolejno (`x1`, `y1`), (`x2`, `y2`), (`x3`, `y3`).
- `circumference(self)` – zwraca obwód trójkąta `self`.
- `area(self)` – zwraca pole trójkąta `self`.
- `contains_point(self, x, y)` – zwraca `True`, jeśli trójkąt `self` zawiera punkt o współrzędnych (`x1`, `y1`) i `False` w przeciwnym wypadku.
- `contains(self, other_triangle)` – zwraca `True`, jeśli trójkąt `other_triangle` zawiera się w trójkącie `self`, `False` w przeciwnym wypadku.
- `contained_in(self, other_triangle)` – zwraca `True`, jeśli trójkąt `self` zawiera się w trójkącie `other_triangle`, `False` w przeciwnym wypadku.
- (za 0,25 punktu) `draw(self)` – rysuje i wyświetla brzeg trójkąta `self` z użyciem biblioteki `matplotlib`.

Możesz założyć, że reprezentujemy jedynie trójkąty niezdegenerowane.

[1] Niewątpliwie zbudowanych na polach zamieszkiwalnych.
[2] W zadaniu nie można używać analogicznej, wbudowanej klasy `complex`.
[3] Metody w tym zadaniu przeciążają odpowiednie operatory.